C Y B E R S E C U R I T Y  I N T E R N S H I P

# Week 1 — Security Assessment Report

*Vulnerability Analysis of OWASP NodeGoat Web Application*

| | |
|---|---|
| **Application** | OWASP NodeGoat v1.3.0 |
| **Repository** | github.com/OWASP/NodeGoat |
| **Assessment Type** | Static Code Analysis + Manual Penetration Testing |
| **Assessment Date** | March 11, 2026 |
| **Prepared By** | Hussein Khaed |
| **Classification** | Confidential |

# 1. Executive Summary

This report presents the findings of a Week 1 security assessment conducted on the OWASP NodeGoat application — a deliberately vulnerable Node.js web application used for security training. The application was cloned from GitHub, inspected via static code analysis, and manually tested for common OWASP Top 10 vulnerabilities.

A total of 10 vulnerabilities were identified across critical, high, and medium severity levels. The most critical issues include plaintext password storage, missing authentication security headers, disabled CSRF protection, and a Server-Side Request Forgery (SSRF) vulnerability. These findings demonstrate real-world risks that must be addressed before any production deployment.

| ID | Vulnerability | Severity | OWASP Category |
|---|---|---|---|
| **V-01** | Plaintext Password Storage | **Critical** | A07: Identification & Auth |
| **V-02** | Server-Side Request Forgery | **Critical** | A10: SSRF |
| **V-03** | Stored XSS via Memo Input | **High** | A03: Injection |
| **V-04** | Missing Security Headers (Helmet) | **High** | A05: Security Misconfiguration |
| **V-05** | CSRF Protection Disabled | **High** | A01: Broken Access Control |
| **V-06** | Session Fixation on Login | **High** | A07: Identification & Auth |
| **V-07** | Sensitive Data Stored Unencrypted | **High** | A02: Cryptographic Failures |
| **V-08** | ReDoS via Regex Backtracking | **Medium** | A06: Vulnerable Components |
| **V-09** | Log Injection / CRLF Injection | **Medium** | A09: Logging Failures |
| **V-10** | Hardcoded Secrets in Config | **Medium** | A05: Security Misconfiguration |

# 2. Scope & Methodology

## 2.1 Application Overview

The target application is OWASP NodeGoat, a Node.js/Express web application backed by MongoDB. It includes the following functional areas:

- User registration and login (session-based authentication)
- User profile management (personal details, SSN, bank account)
- Memo/notes board (user-generated content)
- Research/stock lookup (external HTTP requests)
- Contributions and allocations tracking

## 2.2 Testing Methodology

The following testing techniques were applied during the Week 1 assessment:

- Static Code Analysis — All route handlers, DAO files, and configuration files were manually inspected for insecure coding patterns.
- Manual Injection Testing — Login and form fields were tested with payloads such as:
  - XSS payload: <script>alert('XSS');</script> in text input fields
  - SQL/NoSQL Injection: admin' OR '1'='1 in username/password
- Configuration Audit — server.js, config/env/all.js, and package.json were reviewed for misconfigurations, hardcoded secrets, and outdated dependencies.
- Browser Developer Tools — Network tab and cookie inspector used to review session tokens, response headers, and cookie flags.

## 2.3 Tools Used

| Tool | Purpose |
|---|---|
| **OWASP ZAP** | Automated web application vulnerability scanner — active and passive scan modes |
| **Browser DevTools** | Inspect DOM, HTTP headers, cookies, and simulate manual XSS payloads |
| **Manual Code Review** | Static analysis of all JavaScript source files in /app/routes and /app/data |
| **npm audit** | Dependency vulnerability scanner for known CVEs in package.json |
| **Nmap (planned Week 3)** | Network port and service discovery for attack surface mapping |

## 3. Detailed Vulnerability Findings

| [V-01] Plaintext Password Storage | |
|---|---|
| **Severity** | **Critical** |
| **CVSS Score** | 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H) |
| **Description** | User passwords are stored directly in MongoDB as plaintext strings. The bcrypt hashing code is present in the codebase but commented out. If the database is compromised, all user credentials are immediately exposed. |
| **Affected Component** | app/data/user-dao.js — addUser() and validateLogin() functions |
| **Evidence / Test** | In addUser(): password field is stored as-is from req.body. The fix line '// password: bcrypt.hashSync(password, bcrypt.genSaltSync())' is commented out. In validateLogin(): comparison uses '===' string equality instead of bcrypt.compareSync(). |
| **Recommendation** | Uncomment bcrypt.hashSync() in addUser(). Replace string comparison with bcrypt.compareSync() in validateLogin(). Use a work factor of at least 10 (bcrypt default). |

| [V-02] Server-Side Request Forgery (SSRF) | |
|---|---|
| **Severity** | **Critical** |
| **CVSS Score** | 9.1 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N) |
| **Description** | The research route accepts a user-controlled 'url' query parameter and makes a direct outbound HTTP request to it without any validation. An attacker can use this to probe internal network services, read cloud metadata endpoints (e.g., http://169.254.169.254/), or pivot to internal hosts. |
| **Affected Component** | app/routes/research.js — displayResearch() handler |
| **Evidence / Test** | Request: GET /research?url=http://169.254.169.254/latest/meta-data/&symbol=iam — The server will forward this request to the AWS metadata endpoint, potentially leaking IAM credentials. The needle.get() call uses the raw, unsanitized url parameter from req.query. |
| **Recommendation** | Implement a strict allowlist of permitted URL prefixes. Validate and parse the URL before making any outbound request. Block requests to private IP ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16). |

| [V-03] Stored Cross-Site Scripting (XSS) via Memo Input | |
|---|---|
| **Severity** | **High** |
| **CVSS Score** | 7.4 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N) |
| **Description** | The memo submission endpoint accepts arbitrary user input and stores it in MongoDB without any sanitization. When memos are displayed, the content is rendered directly into the HTML page, allowing persistent XSS that affects all users who view the memos page. |
| **Affected Component** | app/routes/memos.js — addMemos() and displayMemos(); app/data/memos-dao.js |
| **Evidence / Test** | Test payload entered in memo field: <script>alert('XSS')</script>. The content is stored verbatim and rendered when any user visits the /memos page. The memo content is passed directly from req.body.memo to the database with no sanitization. |

| | |
|---|---|
| **Recommendation** | Sanitize all user inputs with a library such as DOMPurify (client-side) or sanitize-html (server-side). Implement Content Security Policy (CSP) headers via Helmet.js to prevent inline script execution. |

### [V-04] Missing HTTP Security Headers (Helmet.js Disabled)

| | |
|---|---|
| **Severity** | **High** |
| **CVSS Score** | 7.2 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L) |
| **Description** | Helmet.js is imported in server.js but entirely commented out. This means critical HTTP security headers are absent, including: X-Frame-Options (clickjacking), Content-Security-Policy (XSS), Strict-Transport-Security (HSTS), and X-Powered-By removal (technology fingerprinting). |
| **Affected Component** | server.js — lines with 'const helmet = require("helmet")' and 'app.use(helmet())' are both commented out |
| **Evidence / Test** | curl -I http://localhost:4000 returns no X-Frame-Options, no Content-Security-Policy, no X-Content-Type-Options headers. The 'X-Powered-By: Express' header is present, revealing the framework version to attackers. |
| **Recommendation** | Uncomment 'const helmet = require("helmet")' and 'app.use(helmet())' in server.js. Configure specific helmet options for CSP, HSTS, and frameguard. Call app.disable('x-powered-by') to remove the Express fingerprint. |

### [V-05] CSRF Protection Disabled

| | |
|---|---|
| **Severity** | **High** |
| **CVSS Score** | 7.1 (AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N) |
| **Description** | The csurf middleware for Cross-Site Request Forgery protection is imported but commented out. Any authenticated action (profile update, memo creation, fund transfers) can be triggered by a malicious third-party site that tricks the victim's browser into submitting forged requests. |
| **Affected Component** | server.js — 'const csrf = require("csurf")' is commented out and never applied |
| **Evidence / Test** | A malicious page with <form action='http://localhost:4000/profile' method='POST'> and an auto-submit script can change a victim's profile data if the victim visits the page while authenticated. No CSRF token is required or verified. |
| **Recommendation** | Uncomment the csurf import and apply it as middleware: app.use(csrf()). Add the CSRF token to all HTML forms as a hidden field. Validate the token on every state-changing POST/PUT/DELETE request. |

### [V-06] Session Fixation on Login

| | |
|---|---|
| **Severity** | **High** |
| **CVSS Score** | 7.0 (AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:N) |
| **Description** | The login handler does not regenerate the session ID upon successful authentication. If an attacker can force a pre-known session ID on the victim (via network interception or shared links), they can hijack the authenticated session after login. |
| **Affected Component** | app/routes/session.js — handleLoginRequest() function |

| | |
|---|---|
| **Evidence / Test** | The comment in the code explicitly identifies this: 'Fix the problem by regenerating a session in each login by wrapping the below code as a function callback for req.session.regenerate()'. Currently, req.session.userId is set on the existing session ID without regeneration. |
| **Recommendation** | Wrap the session assignment inside req.session.regenerate(() => { req.session.userId = user._id; res.redirect(...); }). This ensures a fresh session token is issued on every login. |

## [V-07] Sensitive Data Stored Unencrypted

| | |
|---|---|
| **Severity** | **High** |
| **CVSS Score** | 6.8 (AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:N/A:N) |
| **Description** | Sensitive personal information including Social Security Numbers (SSN), dates of birth, and bank account numbers are stored in MongoDB in plaintext. The encryption implementation using Node.js crypto module is present but entirely commented out. |
| **Affected Component** | app/data/profile-dao.js — updateUser() function; the encrypt/decrypt helpers are commented out |
| **Evidence / Test** | Database inspection shows SSN, dob, bankAcc, and bankRouting fields stored as plain strings. The fix code using crypto.createCipheriv() is commented out under the '/* Fix for A6 - Sensitive Data Exposure' block. |
| **Recommendation** | Uncomment and enable the encrypt/decrypt helpers. Apply encryption before storing sensitive fields and decryption before returning them. Ensure the cryptoKey is stored securely in environment variables, not in source code. |

## [V-08] ReDoS via Catastrophic Regex Backtracking

| | |
|---|---|
| **Severity** | **Medium** |
| **CVSS Score** | 5.9 (AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:H) |
| **Description** | The bank routing number validation uses a vulnerable regular expression '/([0-9]+)+#/' which contains nested quantifiers. A specially crafted input string can cause exponential backtracking, consuming 100% CPU and causing a denial of service. |
| **Affected Component** | app/routes/profile.js — handleProfileUpdate(); const regexPattern = /([0-9]+)+\#/ |
| **Evidence / Test** | Input '111111111111111111111111111!' causes exponential backtracking. The code comment explicitly identifies the vulnerability: 'specific type of input may cause it to consume all CPU resources with an exponential time'. The safe fix '/([0-9]+)\#/' is shown but commented out. |
| **Recommendation** | Replace /([0-9]+)+\#/ with the safe alternative /([0-9]+)\#/. For complex regex, consider using the 'safe-regex' npm package to detect catastrophic patterns at test time. |

## [V-09] Log Injection / CRLF Injection

| | |
|---|---|
| **Severity** | **Medium** |
| **CVSS Score** | 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N) |
| **Description** | Failed login attempts are logged using console.log() with the raw, unencoded username from the request. An attacker can inject CRLF characters (\r\n) in the username to forge log entries, potentially hiding malicious activity or injecting false records. |

| | |
|---|---|
| **Affected Component** | app/routes/session.js — handleLoginRequest(); console.log('Error: attempt to login with invalid user: ', userName) |
| **Evidence / Test** | By sending a username such as 'alice\r\nINFO: Admin login successful from 10.0.0.1', the attacker can inject fake log lines. The code comment acknowledges this: 'Fix for A1 - 3 Log Injection - encode/sanitize input for CRLF Injection'. |
| **Recommendation** | Sanitize all logged user input by stripping CRLF characters: userName.replace(/(\r\n\|\r\|\n)/g, '_'). Alternatively use ESAPI encoder: ESAPI.encoder().encodeForHTML(userName) before logging. |

## [V-10] Hardcoded Secrets in Configuration File

| | |
|---|---|
| **Severity** | **Medium** |
| **CVSS Score** | 4.8 (AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N) |
| **Description** | Sensitive cryptographic keys and session secrets are hardcoded as plaintext strings in the configuration file. If the source code is ever made public or committed to a version control repository, these secrets are immediately compromised. |
| **Affected Component** | config/env/all.js |
| **Evidence / Test** | The file contains: cookieSecret: 'session_cookie_secret_key_here' and cryptoKey: 'a_secure_key_for_crypto_here' as static strings. The development config also contains a hardcoded ZAP API key. |
| **Recommendation** | Move all secrets to environment variables. Use a .env file (via the dotenv package) that is excluded from version control via .gitignore. Rotate any secrets that may have been previously committed to a repository. |

# 4. Areas of Improvement

## 4.1 Authentication & Session Management

- Implement bcrypt password hashing with a work factor of 12 or higher
- Regenerate session IDs after successful login to prevent session fixation
- Use secure, HTTPOnly, and SameSite=Strict cookie flags
- Enforce strong password policy (minimum 8 characters, mixed case, numbers)
- Implement account lockout after 5 failed login attempts

## 4.2 Input Validation & Output Encoding

- Sanitize all user-generated content before storage using sanitize-html or DOMPurify
- Apply context-aware output encoding (HTML, URL, JavaScript contexts)
- Validate all inputs on the server side using the validator npm package
- Use parameterized queries and avoid constructing dynamic database queries from user input

## 4.3 Security Headers & Transport Security

- Enable Helmet.js immediately: app.use(helmet())
- Configure Content Security Policy to block inline scripts
- Enable HTTPS using TLS certificates — the HTTPS setup code already exists and should be uncommented
- Set Strict-Transport-Security (HSTS) header to enforce HTTPS

## 4.4 Data Protection

- Encrypt all PII fields (SSN, DOB, bank account) using AES-256 before storing in the database — the crypto code already exists and needs to be enabled
- Use environment variables for all secrets — never hardcode keys in source files
- Add .env to .gitignore to prevent accidental secret exposure via version control

## 4.5 Application Architecture

- Enable CSRF protection via the csurf middleware for all state-changing requests
- Restrict outbound HTTP requests to an allowlist of trusted domains to prevent SSRF
- Replace vulnerable regex patterns with safe alternatives to prevent ReDoS
- Set up a dependency update process — several dependencies (marked 0.3.5, bcrypt-nodejs 0.0.3) have known CVEs

## 5. Risk Summary Matrix

| Vulnerability | Likelihood | Impact | Risk Level | Priority |
|---|---|---|---|---|
| Plaintext Password Storage | High | Critical | **Critical** | **P1** |
| SSRF via Research URL | High | Critical | **Critical** | **P1** |
| Stored XSS — Memos | High | High | **High** | **P1** |
| Missing Security Headers | High | High | **High** | **P2** |
| CSRF Protection Disabled | Medium | High | **High** | **P2** |
| Session Fixation | Medium | High | **High** | **P2** |
| Unencrypted PII Storage | Medium | High | **High** | **P2** |
| ReDoS Regex | Medium | Medium | **Medium** | **P3** |
| Log/CRLF Injection | Medium | Medium | **Medium** | **P3** |
| Hardcoded Secrets | Low | High | **Medium** | **P3** |

# 6. Conclusion & Next Steps

The Week 1 security assessment of the OWASP NodeGoat application has identified 10 vulnerabilities, including 2 critical, 5 high, and 3 medium severity issues. These findings closely mirror the OWASP Top 10 list and provide an excellent learning opportunity for understanding real-world web application security.

Critically, many of the fixes already exist in the codebase but have been commented out for learning purposes. The Week 2 tasks will focus on enabling these fixes and implementing additional hardening measures.

## 6.1 Immediate Actions (Week 2 Focus)

- **Enable bcrypt password hashing — uncomment the fix in user-dao.js**
- **Enable Helmet.js security headers — uncomment in server.js**
- **Enable CSRF protection via csurf middleware**
- **Sanitize memo inputs before storage and rendering**
- **Regenerate session ID on login**

## 6.2 Short-Term Actions (Week 3 Focus)

- Implement JWT-based authentication
- Enable HTTPS with TLS
- Restrict SSRF attack surface via URL allowlist
- Encrypt PII fields in the database
- Move all secrets to environment variables
- Set up winston logging with proper input sanitization

# 7. References


- OWASP Top 10 (2021): https://owasp.org/www-project-top-ten/
- OWASP NodeGoat Repository: https://github.com/OWASP/NodeGoat
- OWASP ZAP Scanner: https://www.zaproxy.org/
- Helmet.js Documentation: https://helmetjs.github.io/
- bcrypt npm package: https://www.npmjs.com/package/bcrypt
- OWASP XSS Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP SSRF Prevention: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- CWE-352 (CSRF): https://cwe.mitre.org/data/definitions/352.html